

版本火车需求管理系统

AI 原生研发竞赛项目汇报

AI原生研发能力攻关赛道

MVP · 端到端 AI Coding · 17天交付

2026 年 5 月

目录

第一章 作品定位与业务背景

第二章 痛点分析

第三章 MVP 能力范围

第四章 AI 原生研发闭环与协作规范

第五章 业务需求说明书

第六章 系统总体架构与技术实现地图

第七章 数据模型与 API 集成设计

第八章 竞赛亮点

第一章 作品定位与业务背景

1.1 作品定位

版本火车需求管理系统是一个面向多系统协同发布场景的研发管理工具。系统用"版本火车"统一承载发布周期、关键节点、团队容量、需求纳版、依赖风险和投产回滚，解决传统需求管理工具偏单需求记录、缺少跨系统发布节奏管理的问题。

本作品面向 AI 原生研发能力攻关赛道，重点展示一个从业务需求、方案设计、代码开发、测试验证到工程文档交付的端到端 AI coding 项目。项目不是只在单点功能中使用 AI，而是将 AI 作为持续协作主体，贯穿需求澄清、领域建模、实现定位、测试设计、问题交接和竞赛材料沉淀。

1.2 业务背景

企业系统数量增加后，需求往往跨系统协同交付。多个系统之间存在需求依赖、投产窗口、测试节奏、团队容量和版本范围控制等问题。如果仍使用单系统、单需求、静态记录式管理方式，容易出现以下问题：

问题	影响	本项目应对方式
发布范围不清晰	临近投产频繁加塞	通过纳版机制冻结班次范围
跨系统依赖不透明	需求排期互相阻塞	依赖风险分级提示
团队容量不可见	计划超载但无预警	班次容量快照和使用率监控
关键节点不统一	开发、测试、投产节奏割裂	统一纳版、SIT、UAT、封板、投产节点
决策过程不可追溯	回滚、变更责任不清	状态日志和紧急变更记录

第二章 痛点分析

本章从需求源头到交付结果，层层递进分析多系统协同发布场景下的五大核心痛点。

2.1 业务需求缺乏规范：一句话需求满天飞

业务方提交需求时往往只有一句话描述，如"做个导出功能""优化一下登录"，缺少结构化的业务背景、验收标准、优先级和影响范围。这种"一句话需求"在多系统协同场景下会被放大：

混乱表现	典型场景	后果
需求描述模糊	"优化支付流程"——是提速？改 UI？还是换通道？	开发按自己理解实现，交付后业务说"不是这个意思"
缺少验收标准	需求没有明确的"做完"定义	测试无法判断是否通过，反复返工
优先级随意	业务认为都是 P0，开发无法判断先做哪个	真正紧急的需求被淹没在"都很急"的噪音中
影响范围不清	业务不知道需求涉及几个系统	纳版时才发现跨系统依赖，排期被打乱
变更频繁	需求没有基线，随时口头加要求	开发做到一半被叫停，已完成的代码废弃

根本原因：缺少需求录入的结构化规范和评审机制。业务方习惯用"一句话"表达意图，而现有工具没有强制要求补充业务背景、用户故事、验收标准和优先级，导致需求从源头就是模糊的，后续所有环节都在为这个模糊买单。

2.2 需求变更失控：口头变更、影响不可追溯

需求从评审通过到投产之间，业务方随时可能提出变更。多系统协同场景下，一个需求的变更可能波及多个系统的排期和依赖关系，但变更过程往往缺乏记录和分析：

混乱表现	典型场景	后果
变更无记录	业务在飞书群里说"那个需求改一下"，没有正式变更单	开发按口头要求改了，但测试不知道，按原需求测
影响不可见	变更影响 3 个系统的接口，但只有 1 个系统知道	投产时才发现其他系统没适配，紧急回滚
变更无评估	变更增加 5 人天工作量，但没人提前评估	班次容量超载，其他需求被挤占
变更与原需求脱节	需求改了 3 次，最终版本和最初评审的版本完全不同	投产后业务说"不是我要的"，责任不清

根本原因：缺少结构化的变更管理机制。变更没有标准流程（提出→影响分析→确认→执行），没有工具自动检测变更内容并评估影响范围，导致变更成为项目延期和质量问题的最大隐患之一。

2.3 BizDevOps 落地难：工具链断裂，协同成本高

BizDevOps 理念强调业务、开发、测试的端到端打通，但在企业实际落地中面临多重断裂：

断裂点	具体表现	后果
业务→开发	需求在 Excel/邮件/Jira 之间流转，缺少结构化的跨系统关联	开发拿到的需求缺少依赖上下文，排期凭经验盲估

断裂点	具体表现	后果
开发→测试	测试用例与需求脱节，SIT/UAT 环境切换靠人工协调	测试覆盖率不可度量，封板前才发现遗漏
测试→投产	投产清单手工拼凑，回滚决策缺乏数据支撑	一次投产涉及 5+ 系统时，协调成本指数级上升

根本原因：现有工具（Jira、Confluence、GitLab）各自解决单点问题，缺少一个统一承载"发布节奏"的容器。业务关心"需求何时上线"，开发关心"代码何时合入"，测试关心"什么时候可以开始测"——三者的时间线没有对齐机制。

2.4 敏捷实践变形：迭代快了，交付却没快

许多团队引入了 Scrum/Kanban，但敏捷的承诺——更快交付价值——并未兑现：

变形现象	典型场景	根因
迭代空转	每 Sprint 开站会、评审会、回顾会，但需求跨系统依赖导致无法按时交付	缺少跨团队的依赖可视化和容量规划
需求积压	Backlog 越来越长，PO 分不清哪些需求能进当前班次	缺少"纳版"机制来冻结和承诺发布范围
紧急变更失控	每个班次封板后仍有 3~5 个紧急变更插入，打乱测试计划	紧急变更没有分级治理和影响评估机制
回顾会流于形式	问题反复提出但无法闭环，因为缺少跨系统的度量数据	没有统一的进度视图和风险聚合

核心矛盾：敏捷框架假设团队是自组织的，但多系统协同发布场景下，"团队"的定义已经超出单个 Scrum Team。缺少跨团队的节奏同步机制，敏捷就退化成了"开会更频繁"。

2.5 版本管理混乱：谁在哪个版本、什么时候上线

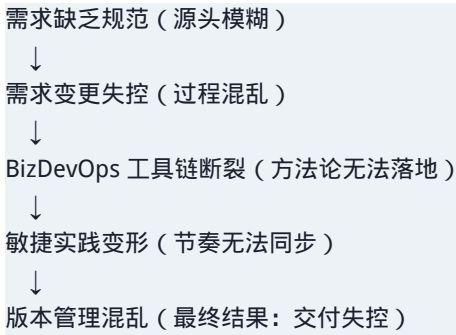
当企业系统数量从个位数增长到数十个，版本管理从"简单问题"变成"系统性风险"：

混乱表现	影响
发布范围不透明	临近投产还在加塞需求，测试来不及，质量风险高
跨系统依赖不可见	A 系统依赖 B 系统的接口变更，但 B 系统延期了，A 系统投产当天才发现
团队容量黑盒	某个系统已经被分配了超过承载力的需求，但没有人提前预警
关键节点不统一	开发说"代码合了"、测试说"还没测完"、投产窗口还没确认，三方节奏割裂
回滚决策困难	投产后发现问题，但不知道回滚会影响哪些系统的哪些需求，不敢轻易回滚

本质问题：版本管理不是"记录版本号"，而是管理一组系统在时间轴上的协同节奏。传统工具只管"单个需求的版本"，不管"一组系统的发布节奏"。

2.6 痛点总结：为什么需要"版本火车"

上述痛点形成了一个层层递进的因果链条：



共同根源是：缺少一个以"发布节奏"为核心的管理容器，且缺少 AI 赋能来提升需求质量和变更管理效率。

- 需求缺乏规范 → 需要 AI 智能审查，在需求提交前自动检查质量，从源头消除模糊
- 需求变更失控 → 需要 AI 驱动的变更检测和影响分析，让变更可追溯、可评估
- BizDevOps 的断裂 → 需要统一的时间线对齐业务、开发、测试、投产
- 敏捷的变形 → 需要跨团队的节奏同步机制（纳版、封板、投产窗口）
- 版本管理的混乱 → 需要一个承载多系统发布范围的容器（版本火车 + 班次）

版本火车需求管理系统正是围绕这个核心洞察设计的：用"火车"统一承载发布周期，用"班次"冻结发布范围，用"纳版"建立需求承诺机制，用"关键节点"对齐多方节奏，用 AI 赋能需求审查和变更管理，实现从需求到投产的全流程智能化。

第三章 MVP 能力范围

MVP 聚焦六个核心能力：

模块	核心能力	实现位置
需求池管理	需求创建、编辑、评审、取消、变更、子状态流转	modules/requirements
版本火车与班次管理	火车容器、班次、搭载系统、容量快照、关键节点、纳版、移除、投产、回滚	modules/trains
AI 智能纳版	基于规则引擎和 AI 解释的排期建议、容量分析、依赖风险提示	modules/smart-onboard

模块	核心能力	实现位置
AI 需求审查	提交评审前 AI 自动检查需求质量（格式、完整性、验收条件、歧义检测），评分 + 改进建议	modules/ai-review
需求变更智能体	Coze 智能体自动检测需求变更、生成影响分析、创建变更单，飞书消息卡片通知和确认	modules/change-request + Coze
统一仪表盘	统计卡片、待办事项、关键节点倒计时、班次进度	pages/dashboard

第四章 AI 原生研发闭环与协作规范

4.1 端到端 AI 研发流程

本项目的 AI 原生研发不是"让 AI 写几段代码"，而是将 AI 作为持续工程协作者，参与需求澄清、文档设计、实现定位、代码生成、测试验证、问题交接、项目记忆和竞赛材料沉淀。

- 需求智能分析：从总 PRD、Task PRD、用户故事和详细设计中抽取领域语言和模块边界
- 方案设计：围绕权限矩阵、状态机、数据模型、接口设计和页面原型进行设计冻结
- 代码实现：在 TypeScript monorepo 中由 AI 按模块实现前端、后端、共享类型和数据库模型
- 测试验证：通过后端 Vitest、前端 Vitest、智能纳版测试案例和人工演示路径验证
- 交接沉淀：通过 handoff、工作小结、代码结构映射和 issue 化工程文档保持上下文连续
- 竞赛材料封装：将工程事实转化为评审可读的项目级文档、演示脚本和证据包

4.2 SDD 规则文档体系

项目采用 SDD (Specification-Driven Development) 规则文档体系，以 AGENTS.md 为唯一入口和最高优先级阅读文档。规则层包含四个核心规范文档：AI 协作规范、编码规范、安全规范和项目实现规划。AI 修复 Bug 时通过代码结构映射文档快速定位，不再需要全仓库无序搜索。

4.3 核心协作原则

原则	工程含义	项目落地
先设计后编码	未冻结设计前不进入实现	AGENTS.md 要求设计审核通过后才能编码
文档代码双向同步	设计变更和代码变更必须反向更新	PRD、设计方案、测试案例、版本记录共同维护
增量验证	每完成一个功能点立即验证	Task/US 粒度测试和 handoff 记录

原则	工程含义	项目落地
状态机先行	涉及流程先画状态流转	需求状态机、班次状态机、紧急变更审批
权限矩阵先行	涉及角色先定义权限边界	PERMISSION_MATRIX 与 PRD 权限表对应
上下文可接手	每次阶段性工作可由新 AI 继续	handoff、工作小结、代码结构映射、issue 化文档

4.4 Coze 智能体集成

本项目通过 Coze 平台集成了三个 AI 能力，构成完整的智能体流水线：

智能体	Coze 类型	核心能力
T3 智能纳版	工作流	需求列表 + 容量 → 纳版建议 + AI 解释
T6 AI 需求审查	工作流	需求表单 → 评分 + 问题 + 建议 + 验收条件
T5 需求变更智能体	智能体 Bot	飞书群聊 → 检测变更 → 插件查询 → 变更单 + 卡片

4.5 项目进度总览

阶段	实际日期	状态	关键产出
Task 0 基础框架	05-08 ~ 05-10	完成	Schema、认证、RBAC、安全规范
Task 1 需求池管理	05-10 ~ 05-15	完成	US1.1~US1.10 全部需求 CRUD + 275 测试
Task 2 版本火车管理	05-11 ~ 05-19	完成	US2.1~US2.9 火车/班次/状态联动
Task 3 智能纳版	05-20 ~ 05-24	完成	Coze AI + 规则引擎 + 可装则装
Task 4 仪表盘重构	05-22 ~ 05-24	完成	仪表盘 UI + 双月历视图
Task 5 需求变更智能体	05-28 ~ 05-29	完成	插件 API + Coze Bot + 飞书卡片
Task 6 AI 需求审查	05-29	完成	本地 7 项 + Coze AI + Modal Tabs

4.6 质量保证

维度	手段	结果
单元测试	Vitest + Supertest + Testing Library	275 用例 100% 通过

维度	手段	结果
安全审计	RT-安全规范清单（无明文密钥、参数化查询、日志脱敏）	全部达标
代码规范	coding-standards.md（类型优先、分层清晰、注释完整）	遵守
权限校验	RBAC 中间件 + service 层二次校验	前端 + 后端双控
人工审核	设计审核、代码审核、Git 操作确认 3 个关卡	严格执行
AI 降级	Coze 不可用时自动回退（本地规则审查、规则引擎）	T3/T6 均实现

第五章 业务需求说明书

5.1 目标用户与角色

角色	主要职责
业务 BA	录入需求、维护业务归属、发起评审、UAT 通过、查看自己的待办
产品经理	辅助录入和编辑需求，参与需求信息完善
项目经理	评审需求、审批紧急变更、查看项目级待办
技术经理	推进开发完成、参与子状态管理
测试经理	推进 SIT 通过、审批紧急变更
火车管理员	创建火车和班次、配置搭载系统、纳版、移除、投产、回滚、完成班次
超级管理员	系统级兜底权限

5.2 核心业务流程

5.2.1 需求从录入到纳版

- BA 或产品经理创建草稿需求
- 补齐系统、BA、PM、优先级、点数和依赖
- 发起评审 → 项目经理评审通过后进入已就绪
- 火车管理员在班次中选择已就绪需求
- 系统执行容量和依赖风险检查

- 火车管理员确认纳版，需求进入已纳版和开发中子状态

5.2.2 班次从规划到投产

- 火车管理员创建版本火车并配置搭载系统
- 创建班次，系统生成关键节点并保存容量快照
- 班次中纳入需求，占用对应系统容量
- 按开发、SIT、UAT、封板推进
- 封板后进入严格变更治理
- 投产节点后确认需求投产，所有需求处理完成后完成班次

5.2.3 变更分级管理

系统支持三级变更管理机制，根据变更时机和影响程度采取不同治理策略：

变更级别	触发时机	处理流程	审批要求
普通变更	封板前，需求处于已就绪或已纳版状态	需求变更 → 退回草稿 → 重新评审	无需审批，BA 自助操作
紧急变更	封板后、投产前，已纳版需求需要调整	紧急变更申请 → 审批通过 → 退回草稿 → 重新评审	需项目经理或测试经理审批
强制变更	投产窗口临近，必须立即处理的变更	线下决策 → 系统补录变更单 → 关联审计日志	线下决策，系统记录留痕

关键设计点：封板是变更管理的分水岭。封板前 BA 可自主变更需求；封板后任何变更都必须经过审批流程，确保变更可追溯、可评估、可审计。

5.2.4 AI 智能体功能

系统集成三个 AI 智能体，覆盖需求审查、智能排期、变更检测全生命周期：

5.2.4.1 智能纳版（T3）

功能定位：为火车管理员提供 AI 辅助的纳版决策建议，不直接改变业务状态。

工作流程：

- 前端选择候选需求列表，调用 /api/smart-onboard/suggest
- 后端组装班次容量、系统负载、需求优先级、依赖关系等上下文
- 调用 Coze 工作流 SMART_ONBOARD，AI 生成纳版建议和解释
- 前端展示建议面板，包含：推荐纳版列表、容量占用预测、依赖风险提示
- 火车管理员人工确认后，执行正式纳版操作

降级策略：Coze 不可用时，前端提示"AI 服务暂不可用，请人工判断"，不影响核心功能。

5.2.4.2 智能需求审查（T6）

功能定位：BA 提交评审前的自助质量检查工具，识别需求描述中的潜在问题。

检查维度：

检查项	说明	权重
格式规范	标题长度、描述完整性、必填字段	20%
业务背景	是否说明用户场景和业务价值	20%
验收条件	是否有明确的可测试验收标准	25%
歧义检测	是否存在模糊词汇（优化、完善等）	20%
依赖说明	是否明确跨系统依赖和影响范围	15%

输出结果：综合评分（本地规则 70% + AI 评分 30%）、问题列表、改进建议、优化后的验收条件。

降级策略：Coze 超时或失败时，仅使用本地 7 项规则计算评分，确保功能可用。

5.2.4.3 需求变更助手（T5）

功能定位：通过飞书群聊自动检测需求变更意图，生成变更单和影响分析。

工作流程：

- 业务在飞书群 @需求变更助手，描述变更内容
- Coze Bot 解析对话，提取变更意图、涉及需求、变更内容
- 调用后端插件 API 查询需求详情、依赖关系、当前状态
- AI 生成影响分析：工作量变化、进度影响、风险等级、关联系统
- 推送飞书交互卡片，展示变更单详情，支持一键确认或取消

核心价值：将口头变更转化为结构化变更单，确保变更可追溯、影响可评估。

5.3 明确排除清单

排除项	说明
企业 SSO	MVP 仅保留内置演示用户，SSO 后续版本支持
用户和组织管理	不做完整通讯录、组织架构、角色授权后台
研发任务拆解	已纳版后的研发协同任务暂用子状态模拟
AI 自动纳版	AI 不直接改变业务状态，必须人工确认
通知中心	飞书、邮件、站内红点暂不纳入当前 MVP

排除项	说明
AI 审查替代人工评审	AI 审查是 BA 自助质量检查工具，不参与评审决策

第六章 系统总体架构与技术实现地图

6.1 系统架构分层

系统采用分层架构设计，从用户界面到数据存储共六层，清晰划分职责边界：

层级	技术	说明
用户界面层	React + Ant Design + Zustand	5 个核心页面模块，通过 Axios 调用后端
API 网关层	Fastify	统一入口，提供 CORS/JWT/RBAC/Swagger
业务逻辑层	TypeScript 模块	6 个业务模块，按领域拆分
数据访问层	Prisma ORM	类型安全数据库操作，8 个核心模型
数据存储层	PostgreSQL 16	关系型数据库
Coze AI 平台	Coze 工作流/智能体	3 个 AI 能力，独立部署在 api.coze.cn

6.2 AI 智能体流水线架构

本项目通过 Coze 平台集成了三个 AI 能力，构成完整的智能体流水线。选择 Coze 的理由：可视化工作流编排、SDK 原生支持 SSE stream、内置 debug_url、支持自定义 API 插件 + 飞书渠道、零运维成本。

智能体	Coze 类型	核心能力
T3 智能纳版	工作流	需求列表 + 容量 → 纳版建议 + AI 解释
T6 AI 需求审查	工作流	需求表单 → 评分 + 问题 + 建议 + 验收条件
T5 需求变更智能体	智能体 Bot	飞书群聊 → 检测变更 → 插件查询 → 变更单 + 卡片

6.3 后端模块划分

模块	路径	职责	Task
auth	modules/auth	登录、当前用户、开发 seed	T0

模块	路径	职责	Task
systems	modules/systems	系统列表、系统成员查询	T0
requirements	modules/requirements	需求 CRUD、评审、变更、统计、待办	T1
trains	modules/trains	火车、班次、容量、纳版、投产、回滚	T2
smart-onboard	modules/smart-onboard	AI 智能纳版建议与确认	T3
requirement-review	modules/requirement-review	AI 需求审查（本地规则 + Coze）	T6
common/coze	common/coze	Coze SDK 封装（工作流/流式处理）	T3/T6
common/middleware	common/middleware	JWT 鉴权、RBAC	T0

6.4 前端路由与组件

前端采用 React SPA 架构，核心页面包括：登录页、仪表盘、需求管理（列表/创建/详情/编辑）、版本火车管理（列表/创建/详情/编辑）、班次列表、班次详情、系统管理、日历页。关键组件包括：AuthGuard 路由守卫、RequirementForm 需求表单（含 AI 审查 Modal）、TrainForm 火车表单、ScheduleCalendar 单班次月历视图、CalendarView 双月历视图、SmartOnboardSuggestion 智能纳版建议面板。

6.5 技术栈总览

层级	技术	版本	说明
前端框架	React + TypeScript	18.x	SPA 应用
UI 组件库	Ant Design	5.x	企业级中后台组件
状态管理	Zustand	4.x	轻量级全局状态
构建工具	Vite	5.x	快速 HMR
HTTP 客户端	Axios	1.x	请求/响应拦截 + 超时 180s
后端框架	Fastify + TypeScript	4.x	高性能 Node.js 框架
ORM	Prisma	5.x	类型安全数据库操作
数据库	PostgreSQL	16	关系型数据库

层级	技术	版本	说明
鉴权	JWT (fastify-jwt)	—	Bearer Token
日志	pino + pino-roll	—	结构化日志 + 按天轮转
测试	Vitest + Supertest + Testing Library	—	前后端单元/集成测试
AI 能力	Coze API (@coze/api SDK)	—	工作流/智能体 + 流式响应
包管理	pnpm (workspace)	≥8	Monorepo 包管理

6.6 应用启动流程

后端应用启动时按以下流程初始化：

```
main.ts 启动
→ 加载 .env 环境变量
→ 创建 Fastify 实例
→ 注册插件 (CORS / JWT / RateLimit / Swagger)
→ 注册中间件 (authenticate / RBAC)
→ 注册日志 (pino + pino-roll → apps/server/logs/)
→ 初始化 Coze 客户端 (COZE_API_KEY + Workflow IDs)
→ 注册业务路由 (auth / requirements / trains / smart-onboard / review / systems)
→ listen(3000)
```

6.7 共享包职责

`packages/shared` 是前后端领域契约中心，统一维护类型定义和常量：

文件	职责	Task
constants/index.ts	角色、状态、权限矩阵、显示标签	T0~T4
constants/error-codes.ts	错误码注册表	T0
types/api.ts	通用 API 响应和分页类型	T0
types/auth.ts	登录、用户安全视图	T0
types/requirement.ts	需求 DTO 和列表详情类型	T1
types/review.ts	审查结果类型 (ReviewIssue/ReviewResult)	T6
types/train.ts	火车、班次、纳版相关类型	T2
types/smart-onboard.ts	智能纳版输入输出	T3

文件	职责	Task
types/dashboard.ts	仪表盘聚合数据	T4

6.8 关键运行时链路

6.8.1 需求创建链路

RequirementForm

- services/requirement.ts
- POST /api/requirements
- requirementRoutes
- createRequirement()
- Prisma Requirement / StatusLog

6.8.2 AI 需求审查链路

RequirementForm (AI审查按钮)

- handleAIReview() 校验必填项
- setReviewLoading(true) + showReviewModal
- services/requirement.ts:reviewData()
- POST /api/requirements/review (timeout: 180s)
- requirement-review/index.ts (鉴权 + RBAC)
- reviewRequirementData()
 - ├─ runLocalReview() 7项本地规则 (同步)
 - └─ runAiReview() Coze 工作流 (异步 60-110s)
 - ├─ mapPriorityToChinese() 枚举 → 中文
 - ├─ mapReqTypeToChinese()
 - ├─ mapSourceChannelToChinese()
 - └─ coze.runWorkflow(REQUIREMENT_REVIEW, {...})
 - ├─ 成功 → issues/suggestions/score
 - + optimizedTitle/Description/acceptanceCriteria
 - └─ 失败 → 降级, 返回空, 不阻塞
- 合并结果: localScore×70% + aiScore×30%
- HTTP 200 → ApiResponse
- setReviewResult() → Modal Tabs 展示

6.8.3 班次纳版链路

schedule-detail.tsx

- services/train.ts
- POST /api/trains/schedules/:scheduleId/onboard
- operationsRoutes
- onboardRequirements()
- Requirement.status = ONBOARDED
- TrainSystemSnapshot.usedPoints 更新
- StatusLog 写入

6.8.4 智能纳版链路

SmartOnboardSuggestion

- services/smart-onboard.ts
- POST /api/smart-onboard/suggest
- generateOnboardSuggestions()
 - ├─ getScheduleCapacity() 班次容量
 - ├─ getRequirementsForAI() 需求列表
 - ├─ detectCycleDependencies() 循环依赖检测
 - └─ coze.runWorkflow(SMART_ONBOARD, {...})
 - ├─ trainSchedule: { name, systems }
 - └─ selectedRequirements: [...]
- 合并 AI结果 + 后端容量计算
- 前端展示建议

6.9 工程注意事项

- 项目根要求 Node >= 18、pnpm >= 8
- 日志统一输出到 `apps/server/logs/` (pino-roll 按天轮转 + tee 实时双写)
- Coze SDK 参数不要双重嵌套：SDK 已自动包装 `{ parameters: {...} }`，代码直接传参即可
- 前端 AI 审查超时已配置为 180s (Coze 实际响应 60-110s)
- 测试脚本统一放在 `apps/server/scripts/` 目录

第七章 数据模型与 API 集成设计

7.1 数据模型总览

用户与系统: User, System, SystemMember
需求池: Requirement, RequirementDependency, StatusLog, EmergencyChange
版本火车: Train, TrainSystem, TrainSchedule, TrainSystemSnapshot, TrainScheduleKeyDate

7.2 核心模型说明

7.2.1 用户与系统

Model	作用	关键约束
User	演示用户和角色载体	username、email 唯一；API 响应不返回 password
System	业务系统主数据	name 唯一
SystemMember	用户在系统中的职责	systemId + userId + role 唯一

MVP 中用户和系统由 seed 数据预置，不提供完整组织架构和用户管理后台。

7.2.2 需求池

Model	作用	关键字段
Requirement	需求主体	reqCode、title、status、subStatus、storyPoints、scheduleId
RequirementDependency	前置依赖	dependantId、dependencyId
StatusLog	状态和操作审计	operationType、fromStatus、toStatus、reason
EmergencyChange	封板后紧急变更审批	urgency、status、approvalStep、approverId

需求编号使用 REQ-{年份}-{4位序号} 格式。需求状态和子状态由 shared 枚举统一定义。

7.2.3 版本火车与班次

Model	作用	关键字段
Train	版本火车容器	name、description、version
TrainSystem	火车级系统配置	systemId、capacityPoints、人员字段
TrainSchedule	班次	status、startDate、boardingDate、sitDate、uatDate、lockdownDate、releaseDate
TrainSystemSnapshot	班次容量快照	trainScheduleId、systemId、capacityPoints、usedPoints
TrainScheduleKeyDate	自定义关键日期	name、date

关键设计点：班次创建时复制火车系统配置到 TrainSystemSnapshot。后续调整火车级容量不影响已创建班次的历史容量快照。

7.3 API 响应约定

统一 API 响应格式：

```
成功响应: { success: true, data: {} }
分页响应: { success: true, data: { list: [], total: 0, page: 1, pageSize: 20 } }
错误响应: { success: false, message: "无权限", code: "PERMISSION_DENIED" }
```

7.4 API 分组

7.4.1 需求相关 API

API	说明
GET /api/requirements	需求列表分页筛选
POST /api/requirements	创建需求
GET /api/requirements/:id	需求详情
PATCH /api/requirements/:id	编辑需求
POST /api/requirements/:id/submit-review	发起评审
POST /api/requirements/:id/review-pass	评审通过
POST /api/requirements/:id/review-reject	评审拒绝
POST /api/requirements/:id/change	需求变更
POST /api/requirements/:id/emergency-change	紧急变更

7.4.2 火车与班次 API

API	说明
GET /api/trains	火车列表
POST /api/trains	创建火车
GET /api/trains/:id	火车详情
POST /api/trains/:trainId/schedules	创建班次
GET /api/trains/:trainId/schedules/:scheduleId	班次详情
POST /api/trains/schedules/:scheduleId/onboard	确认纳版
POST /api/trains/schedules/:scheduleId/requirements/:id/remove	从班次移除
POST /api/trains/schedules/:scheduleId/requirements/:id/release	确认投产
POST /api/trains/schedules/:scheduleId/requirements/:id/rollback	回滚

7.4.3 智能纳版 API

API	说明
POST /api/smart-onboard/suggest	生成智能纳版建议

API	说明
POST /api/smart-onboard/confirm	确认并执行纳版
POST /api/requirements/review	AI 需求审查

7.5 权限与鉴权

所有非公开 API 都应通过 `fastify.authenticate` 校验 JWT。涉及业务操作的 API 还应使用 `rbacMiddleware(Operation.xxx)` 校验角色权限。前端按钮隐藏只用于体验，不能作为安全边界。

操作类型	权限枚举
创建需求	Operation.CREATE_REQ
编辑需求	Operation.EDIT_REQ
评审需求	Operation.REVIEW_REQ
管理火车/班次	Operation.MANAGE_TRAIN
回滚	Operation.ROLLBACK_TRAIN

7.6 前后端集成原则

- 页面不直接拼接后端域名，统一使用 `services/api.ts`
- 通用业务 API 封装在 `services/requirement.ts`、`services/train.ts`、`services/system.ts`、`services/smart-onboard.ts`
- 跨端 DTO 和枚举必须从 `@release-train/shared` 引入
- 列表接口使用分页，避免一次性返回无限数据
- 关键业务操作由后端 `service` 事务化处理并写入审计日志

第八章 竞赛亮点

7.1 全流程 AI 驱动

项目以 AI coding 为主线完成需求拆解、设计、实现、测试和文档沉淀。AGENTS.md 明确规定了设计审核、编码审核、增量验证、文档反向同步和 Git 操作规范，使 AI 协作过程具备工程约束。

7.2 领域语言驱动工程实现

CONTEXT.md 将"版本火车、纳版、团队容量、依赖风险、需求变更、紧急变更、投产、回滚"等术语固化为项目语言，避免 AI 在后续实现中误用概念。

7.3 可解释 AI 智能纳版与全流程 AI 赋能

智能纳版不让 AI 直接决定业务结果，而是由确定性规则引擎产生建议，再由 AI 解释容量、优先级和依赖风险。在此基础上，AI 能力进一步延伸到需求全生命周期：

- AI 需求审查 (T6)：提交评审前自动检查需求质量，覆盖格式规范、描述完整性、验收条件、歧义检测等维度，给出评分和改进建议，从源头提升需求质量
- AI 需求变更智能体 (T5)：通过 Coze + 飞书集成，自动检测需求变更、生成影响分析 (工作量/进度/风险)，创建变更单并推送飞书消息卡片，让变更可追溯、可评估
- AI 智能排期：基于规则引擎的排期建议，结合 AI 解释的容量分析和依赖风险提示

这种设计兼顾智能辅助、可控性和审计要求。

7.4 安全研发和权限治理

项目内置 JWT 认证、RBAC 权限矩阵、Fastify schema 参数校验、Prisma 参数化查询、日志脱敏和状态审计，符合竞赛对安全研发和非功能性需求的加分方向。

7.5 可接手的 AI 工程资产

项目沉淀了 PRD、Task 文档、详细设计、代码结构映射、handoff、测试账号、运行手册和工程 issue。后续 AI 可以基于这些资产继续迭代，而不是重新理解项目。